

CUSTOMER LIFETIME VALUE ANALYSIS

A Telecom Customer Retention & Value Segmentation Study

Tools used: Python (Pandas) · SQL (BigQuery) · Power BI

Dataset source: Kaggle — Telco Customer Churn

1. Project Overview

Customer Lifetime Value (CLTV) tells a business which customers are worth the most over the course of their relationship with the company — and which ones are at risk of leaving before they generate meaningful revenue. This project analyzes a telecom provider's customer base to answer a core business question: who are our most valuable customers, what characteristics drive high CLTV, and which segments should retention efforts prioritize?

The dataset combines customer demographics, subscription and service details, billing information, and churn outcomes for 7,043 customers. Each customer already carries a pre-calculated CLTV score; the goal of this analysis was to segment customers by that score, identify the behavioral and contractual drivers behind high vs. low CLTV, and translate those drivers into retention recommendations.

The workflow follows a standard analytics pipeline: data cleaning and feature engineering in Python, business-question analysis in SQL (BigQuery), and an interactive Power BI dashboard for stakeholder-facing exploration.

2. Dataset Summary

Rows: 7,043 customers **Columns:** 33 (raw) → 32 (after cleaning)

Key Feature Groups

- Customer demographics: gender, senior citizen status, partner, dependents, location (city, state, zip)
- Account details: tenure (months), contract type, paperless billing, payment method
- Services subscribed: phone, internet type, online security, streaming TV/movies, tech support, etc.
- Billing: monthly charges, total charges
- Outcome fields: churn label/value/score, CLTV, churn reason

Data Quality Notes

- No duplicate rows found.
- Churn Reason had 5,174 missing values — expected, since this field is only populated for customers who actually churned (1,869 did).
- Total Charges was stored as text and required conversion to numeric before aggregation.

3. Exploratory Data Analysis & Cleaning (Python)

Data was loaded and explored in Pandas to understand structure, detect data quality issues, and prepare it for SQL analysis.

3.1 Initial load and structure check

```
import pandas as pd
raw_df=pd.read_excel('///content/Telco_customer_churn.xlsx')

df=raw_df.copy()

df.head(5)
```

	CustomerID	Count	Country	State	City	Zip Code	Lat	Long	Latitude	Longitude	Gender	...	Contract	Paperless Billing	Payment Method	Monthly Charges	Total Charges	Churn Label	Churn Value	Churn Score	CLTV	Churn Reason
0	3668-QPYBK	1	United States	California	Los Angeles	90003	33.964131	-118.272783	33.964131	-118.272783	Male	...	Month-to-month	Yes	Mailed check	53.85	108.15	Yes	1	86	3239	Competitor made better offer
1	9237-HQITU	1	United States	California	Los Angeles	90005	34.059281	-118.30742	34.059281	-118.307420	Female	...	Month-to-month	Yes	Electronic check	70.70	151.65	Yes	1	67	2701	Moved
2	9305-CDSKC	1	United States	California	Los Angeles	90006	34.048013	-118.293953	34.048013	-118.293953	Female	...	Month-to-month	Yes	Electronic check	99.65	820.5	Yes	1	86	5372	Moved
3	7892-POOKP	1	United States	California	Los Angeles	90010	34.062125	-118.315709	34.062125	-118.315709	Female	...	Month-to-month	Yes	Electronic check	104.80	3046.05	Yes	1	84	5003	Moved
4	0280-XJGEX	1	United States	California	Los Angeles	90015	34.039224	-118.266293	34.039224	-118.266293	Male	...	Month-to-month	Yes	Bank transfer (automatic)	103.70	5036.3	Yes	1	89	5340	Competitor had better devices

5 rows x 33 columns

```
df.info()
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            7043 non-null   object
1   Count                 7043 non-null   int64
2   Country               7043 non-null   object
3   State                 7043 non-null   object
4   City                  7043 non-null   object
5   Zip Code              7043 non-null   int64
6   Lat Long              7043 non-null   object
7   Latitude              7043 non-null   float64
8   Longitude             7043 non-null   float64
9   Gender                7043 non-null   object
10  Senior Citizen        7043 non-null   object
11  Partner               7043 non-null   object
12  Dependents            7043 non-null   object
13  Tenure Months         7043 non-null   int64
14  Phone Service         7043 non-null   object
15  Multiple Lines        7043 non-null   object
16  Internet Service      7043 non-null   object
17  Online Security       7043 non-null   object
18  Online Backup         7043 non-null   object
19  Device Protection     7043 non-null   object
20  Tech Support          7043 non-null   object
21  Streaming TV          7043 non-null   object
22  Streaming Movies      7043 non-null   object
23  Contract              7043 non-null   object
24  Paperless Billing      7043 non-null   object
25  Payment Method        7043 non-null   object
26  Monthly Charges       7043 non-null   float64
27  Total Charges         7043 non-null   object
28  Churn Label           7043 non-null   object
29  Churn Value           7043 non-null   int64
30  Churn Score           7043 non-null   int64
31  CLTV                  7043 non-null   int64
32  Churn Reason          1869 non-null   object
dtypes: float64(3), int64(6), object(24)
memory usage: 1.8+ MB
```

df.head() and *df.info()* confirm 7,043 rows and 33 columns with no unexpected nulls at the column level.

3.2 Missing value check

```
df.isnull().sum() ## cheking for null values
```

CustomerID	0
Count	0
Country	0
State	0
City	0
Zip Code	0
Lat Long	0
Latitude	0
Longitude	0
Gender	0
Senior Citizen	0
Partner	0
Dependents	0
Tenure Months	0
Phone Service	0
Multiple Lines	0
Internet Service	0
Online Security	0
Online Backup	0

Only Churn Reason shows nulls (5,174) — confirmed as expected, since it's only recorded for churned customers.

3.3 Cleaning and de-duplication

```
df['Churn Label'].value_counts() ## we can see the null values in Churn Reasons column is because some customer did not churn

count
Churn Label
No      5174
Yes     1869
dtype: int64

df.duplicated().sum() ## no duplicate values
np.int64(0)

df.columns=df.columns.str.lower().str.replace(' ','_') ## for neat and consistent presentation " SNAKE CASING"
df=df.rename(columns={'customerid':'customer_id'})
df.columns

Index(['customer_id', 'count', 'country', 'state', 'city', 'zip_code',
      'lat_long', 'latitude', 'longitude', 'gender', 'senior_citizen',
      'partner', 'dependents', 'tenure_months', 'phone_service',
      'multiple_lines', 'internet_service', 'online_security',
      'online_backup', 'device_protection', 'tech_support', 'streaming_tv',
      'streaming_movies', 'contract', 'paperless_billing', 'payment_method',
      'monthly_charges', 'total_charges', 'churn_label', 'churn_value',
      'churn_score', 'cltv', 'churn_reason'],
      dtype='object')

df['total_charges']=pd.to_numeric(df['total_charges'],errors='coerce') ## "Type Casting for aggregation purpose from string to float"
df['total_charges']=df['total_charges'].fillna(0)

bins=[0,12,36,df['tenure_months'].max()] ## feature engineering getting customer type from tenure
labels=['New','Established','Loyal']

df['customer_type']=pd.cut(df['tenure_months'],bins=bins,labels=labels,include_lowest=True)
```

- Standardized all column names to snake_case for consistency.
- Confirmed Churn Label and Churn Value carried identical information (encoded differently) — dropped the redundant column.
- Verified Count column was uniformly 1 (one row per customer) and dropped it as redundant with row count.
- Converted Total Charges from text to numeric, filling nulls with 0.

3.4 Feature engineering

```
labels2=['Low','Medium','High'] ##feature engineering segregating customers based on their lifetime value
df['cltv_type']=pd.qcut(df['cltv'],q=3,labels=labels2)

df['churn_label_value']=df['churn_label'].map({'Yes': 1, 'No': 0})
(df['churn_label_value']==df['churn_value']).all() ## so churn_label and churn_value is the same info in diff format, to avoid redundancy i would drop one.
np.True_

df=df.drop(columns=['churn_label']) # so churn_label column dropped
df=df.drop(columns=['churn_label_value']) #temporary column dropped

df['count'].value_counts()

count
count
1      7043
dtype: int64

df['customer_id'].nunique()
7043

df.drop(columns=['count'],inplace=True) # since there are unique values in each row, will drop count column as its redundant

df.to_csv('customer_lifetime_value.csv',index=False)
from google.colab import files
files.download('customer_lifetime_value.csv')
...
```

- Created customer_type by binning tenure_months into New (0–12 mo), Established (12–36 mo), and Loyal (36+ mo).
- Created cltv_type by splitting CLTV into three equal-sized quantile tiers: Low, Medium, High.
- Exported the cleaned dataset and loaded it into BigQuery for SQL-based analysis.

4. Data Analysis Using SQL (BigQuery)

SQL was used against the cleaned BigQuery table to answer three core business questions: who are the highest-value customers, what does the Low-CLTV segment look like, and what factors actually separate High-CLTV customers from Low-CLTV ones. The queries themselves are maintained separately; the results, insight, and recommendation for each are shown below.

4.1 Who are the top-value customers?

Row	customer_id	customer_type	cltv	rnk
1	2511-ALLCS	Established	5998	1
2	7799-DSEWS	Established	5993	2
3	5982-PSMKW	Established	5991	3
4	7622-FWGEW	Loyal	6500	1
5	6024-RUGGH	Loyal	6499	2
6	0383-CLDDA	Loyal	6499	2
7	2683-JXWQQ	Loyal	6495	3
8	5299-RULOA	New	5998	1
9	0455-XFASS	New	5998	1
10	8259-NFJTV	New	5996	2
11	9894-EZEWG	New	5996	2
12	9818-XQCUV	New	5994	3

Insight: The highest-CLTV New customer reaches 5,998 — exactly tied with the top Established customer, and only about 8% below the top Loyal customer (6,500). This is a real nuance on top of the 4.4 finding: tenure clearly drives average CLTV across the base, but it isn't a hard ceiling — a handful of New customers reach nearly peak value almost immediately, without years on the platform behind them.

Recommendation: *Flag the top-ranked New customers for a targeted case review — pull their contract type, payment method, and service bundle.*

4.2 What's the CLTV baseline across key segments?

Before drilling into what separates high- and low-value customers, four baseline queries establish average and total CLTV contribution by customer type, contract, payment method, and internet service.

By contract type

contract ▾	no_of_cust ▾	avg_cltv ▾	total_cltv ▾
Two year	1695	4890.2	8288913
One year	1473	4530.0	6672625
Month-to-month	3875	4136.7	16029745

Insight: Month-to-month customers generate the largest total CLTV in raw dollars simply because there are so many of them (3,875 vs. 1,695 two-year customers) — but their per-customer value is the lowest of the three groups by a clear margin.

Recommendation: *Don't be misled by the total-CLTV column alone — it rewards volume, not quality. Prioritize interventions using average CLTV per segment, where month-to-month is clearly the weakest performer.*

Note: Customer type and payment method showed only modest differences at the baseline level here — the sharper version of both findings, comparing High- vs. Low-CLTV counts directly, appears in Section 4.4. Internet service was also tested at this stage and every other stage of the analysis, and consistently showed no meaningful separation between segments — it's reported once, in Section 4.4, rather than repeated here.

4.3 What does the Low-CLTV segment look like?

Filtering to only Low-CLTV customers and breaking that segment down by city, customer type, contract, internet service, and payment method shows where it's concentrated.

By city (top 10 shown)

city	avg_cltv	no_cust
Los Angeles	2991.28	105
San Diego	2890.92	52
San Jose	2919.4	40
Sacramento	3003.15	33
San Francisco	2988.79	29
Fresno	3005.76	21
Long Beach	3135.48	21
Oakland	2922.68	19
Stockton	3125.17	18
Anaheim	2964.36	14

Insight: Los Angeles holds more than double the Low-CLTV customers of the next-closest city (105 vs. 52), even though average CLTV within the Low tier is nearly identical across all cities.

Recommendation: *Run the first retention pilot in Los Angeles — the offer doesn't need to be city-specific, but concentrating the campaign there reaches the most at-risk customers per dollar spent.*

By customer type

customer_type ▾	avg_cltv ▾	no_cust ▾
New	2987.11	1044
Established	2977.93	900
Loyal	3008.44	404

Insight: New and Established customers together make up about 83% of all Low-CLTV customers; Loyal customers are proportionally rare in this segment even though they're the largest group overall.

Recommendation: *Focus low-CLTV remediation on customers within their first three years — this is where the largest volume of at-risk value sits, and Loyal customers don't need the same intervention.*

By contract type

contract ▾	avg_cltv ▾	no_cust ▾
Month-to-month	2985.5	1709
One year	2993.46	404
Two year	2989.43	235

Insight: Month-to-month customers account for roughly 73% of the entire Low-CLTV segment (1,709 of 2,348), despite average CLTV within the Low tier being almost identical across contract types.

Recommendation: *Contract length — not spending behavior — is the clearest volume lever here. A push to convert month-to-month customers to annual terms should shrink the Low-CLTV segment more than any other single change.*

By payment method

payment_method ▾	avg_cltv ▾	no_cust ▾
Electronic check	2969.6	890
Mailed check	3010.37	689
Credit card (automatic)	2966.37	396
Bank transfer (automatic)	3008.89	373

Insight: Electronic check users are the largest single group within the Low-CLTV segment by a wide margin (890, vs. 689 for the next-highest group), even though average CLTV barely differs by payment method.

Recommendation: *Electronic check is the strongest volume signal available for identifying at-risk customers. Pair the earlier autopay-switch incentive (4.2) specifically with outreach to this group first.*

4.4 What actually separates High-CLTV from Low-CLTV customers?

Comparing High- vs. Low-CLTV customer counts within each segment directly identifies which variables genuinely predict CLTV tier, rather than just correlating with volume.

By customer type

customer_type ▾	High_cltv ▾	Low_cltv ▾
New	515	1044
Established	450	900
Loyal	1381	404

Insight: Loyal customers are more than 3x as likely to be High-CLTV as Low-CLTV. New and Established customers show the reverse pattern — roughly twice as likely to be Low as High.

Recommendation: *Tenure-building programs (renewal perks, milestone rewards) should be the backbone of the CLTV strategy — this is the single clearest split in the entire dataset.*

By contract type

contract ▾	High_cltv ▾	Low_cltv ▾
One year	528	404
Two year	778	235
Month-to-month	1040	1709

Insight: Two-year contract holders are more than 3x as likely to be High-CLTV as Low. Month-to-month is the only contract type where Low-CLTV customers outnumber High-CLTV ones.

Recommendation: *Make contract-length conversion the single top retention priority — pair it with the customer-type finding above, since both point to the same lever: getting customers to commit longer, sooner.*

By payment method

payment_method ▾	High_cltv ▾	Low_cltv ▾
Mailed check	453	689
Bank transfer (automatic)	579	373
Credit card (automatic)	596	396
Electronic check	718	890

Insight: Both automatic payment methods have more High-CLTV than Low-CLTV customers; both check-based methods have the opposite pattern, with electronic check skewing Low most heavily in raw numbers.

Recommendation: *Bundle the payment-method nudge into the same campaign as the contract-length push — a customer who agrees to autopay is a reasonable proxy for one who's also open to a longer commitment.*

By internet service

internet_service ▾	High_cltv ▾	Low_cltv ▾
No	497	521
DSL	805	815
Fiber optic	1044	1012

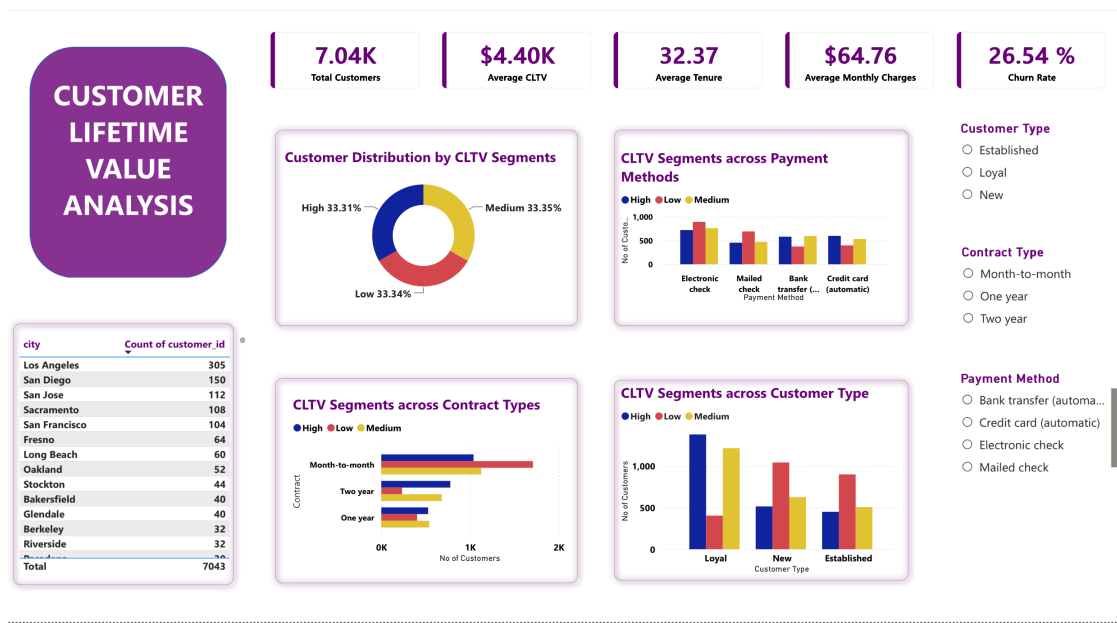
Insight: High- and Low-CLTV counts are almost perfectly balanced within every internet service type — confirming that service type doesn't meaningfully predict CLTV tier on its own.

Recommendation: *Deprioritize internet service type in any predictive or targeting model; contract type and tenure carry essentially all of the signal that service type appeared to carry in the baseline view.*

Section summary: Across all four drivers tested, two stand out clearly — contract length and tenure — while payment method offers a smaller but real secondary signal, and internet service type offers almost none. This ordering directly shapes the priority ranking in Section 6.

5. Dashboard in Power BI

An interactive Power BI dashboard was built to let stakeholders explore CLTV segments live, filtering by customer type, contract type, and payment method rather than reading static query output.



Customer Lifetime Value Analysis dashboard — segment distribution, CLTV by payment method, contract, and customer type, plus a city breakdown and interactive slicers.

7.04K	\$4.40K	32.4 mo	26.54%
Total Customers	Average CLTV	Average Tenure	Churn Rate

- CLTV segments (Low / Medium / High) split almost exactly into equal thirds by design (quantile-based), so the dashboard's value lies in cross-filtering that split by contract, payment method, and customer type.
- Month-to-month and electronic-check filters visibly shift the customer mix toward the Low segment, confirming the SQL findings interactively.
- Los Angeles leads all cities in customer count (305), consistent with it also holding the largest Low-CLTV cluster.

6. Summary

Taken together, these findings point to one underlying story: CLTV in this dataset is driven far more by commitment behavior — contract length, tenure, and payment automation — than by which product a customer buys. That means the highest-leverage retention lever isn't a better product bundle; it's getting customers onto longer contracts and autopay as early in the relationship as possible.

Project repository: github.com/tseringTen/customer-lifetime-value-analysis